

Harmony Hub

Music Content Management Platform

A centralized digital ecosystem for managing songs, artist media, promotional content, and release assets — a smart management and search layer built on top of Google Drive.

Version	1.0
Stack	MERN + Drive
Database	MongoDB
Status	Draft for Review
Date	May 2026

CONTENTS

1. Executive Summary

2. Design Decisions

3. Objectives & Scope

4. System Architecture

5. Upload Flow

6. Search Flow

7. Functional Modules

8. Asset Type Catalogue
9. Data Model (MongoDB)

10. Manual Tagging System

11. Technology Stack

12. UI / UX Concept

13. Development Roadmap

14. Risks & Mitigations

15. Benefits

THE WHY

1. Executive Summary

Music companies produce a large volume of digital assets for every release — master audio, demos, cover art, music videos, social reels, posters, lyrics, contracts, and metadata. In most teams these files are scattered across personal drives, chat threads, and email. The result is duplicate uploads, lost files, slow retrieval, version confusion, and friction whenever teams or partners collaborate.

Harmony Hub solves this. It is an internal web platform that acts as an intelligent management and search layer on top of Google Drive. Drive handles storage, streaming, previews, downloads, and backup, while the platform provides structured organization, rich metadata, instant search, manual tagging,

role-based permissions, controlled sharing, version tracking, and activity analytics.

The outcome is a single source of truth for all music assets: faster retrieval, cleaner organization, lower storage cost, and smoother collaboration across A&R, marketing, production, and partner teams.

1.1 Problem Statement

- Assets are fragmented across many locations with no single index.
- The same file is uploaded multiple times, wasting storage and creating ambiguity.
- Finding a specific asset (e.g. the final master of a song, or a Punjabi promo reel) takes too long.
- There is no reliable version control, so teams accidentally use outdated files.
- Sharing with external partners is manual, insecure, and untracked.

1.2 Proposed Solution

A web application (the management layer) connected to Google Drive (the storage layer) through the Google Drive API. Every file uploaded through the platform is pushed to a structured Drive hierarchy; its Drive file ID plus rich metadata are stored in a MongoDB database that powers fast search, manual tagging, and access control.

FOUNDATIONS

2. Design Decisions

Two choices shape this first version of the platform.

Database — MongoDB

The data model is document-oriented. Each song is a single rich document that embeds its metadata and an array of asset references, which maps naturally to how songs and their assets belong together. This removes join-heavy relational design and simplifies reads.

Tagging — Manual

Editors apply tags by manual approach, or choosing from a controlled tag list plus optional custom tags. This keeps tags accurate and intentional, and keeps the build simple.

Visual-first

A system architecture diagram, an upload flowchart, and a search flowchart are included so the design is easy to understand at a glance. A full asset-type catalogue (Section 8) defines exactly what the platform stores.

BOUNDARIES

3. Objectives & Scope

3.1 Primary Objectives

- Centralize all music-related assets in one organized, searchable hub.
- Enable instant retrieval through metadata, manual tags, and faceted search.
- Manage artists, songs, and their connected assets from a single place.
- Share files securely with internal and external users, with full tracking.
- Maintain version history to eliminate file confusion.

- Improve cross-department collaboration and reduce time spent hunting for media.

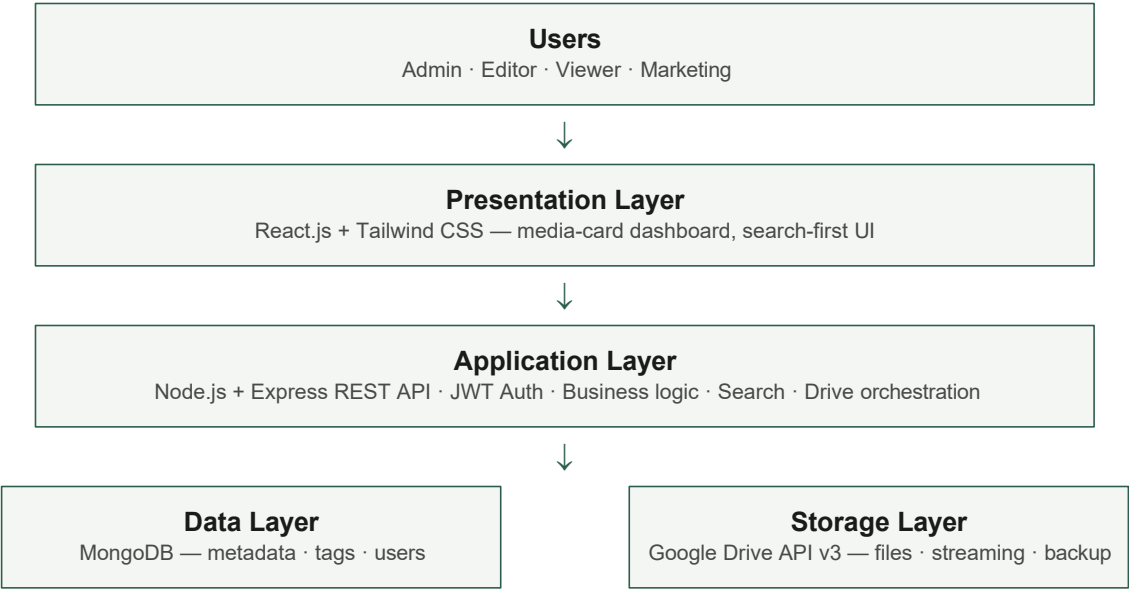
3.2 In Scope (Version 1.0)

- Artist, song, and asset management modules.
- Google Drive integration for upload, storage, preview, and download.
- Manual tagging with a controlled tag list plus custom tags.
- Universal search and discovery.
- Role-based access control and secure sharing links.
- Version management and activity logging.

HOW IT FITS TOGETHER

4. System Architecture

Harmony Hub uses a three-tier architecture with media storage offloaded to Google Drive. The app servers stay lightweight and stateless; Drive provides reliable storage and streaming for large files; MongoDB holds all searchable metadata and the Drive file IDs that link the two.



MongoDB stores the Drive file ID for each asset → links searchable metadata to the actual file.

Figure 1 — Three-tier architecture with Drive as the storage layer and MongoDB as the index.

4.1 Responsibility Split

Google Drive (Storage)	Harmony Hub (Management)
File storage & cloud backup	Content & metadata management
Google Drive (Storage)	Harmony Hub (Management)
Video streaming & image preview	Universal search & discovery
Download handling	Manual tagging (controlled + custom)
Native sharing infrastructure	Roles, permissions & access control
Large-file reliability	Folder organization & version tracking

WRITE PATH

5. Upload Flow

How a file travels from an editor's browser into Drive, with its metadata and tags saved to MongoDB.

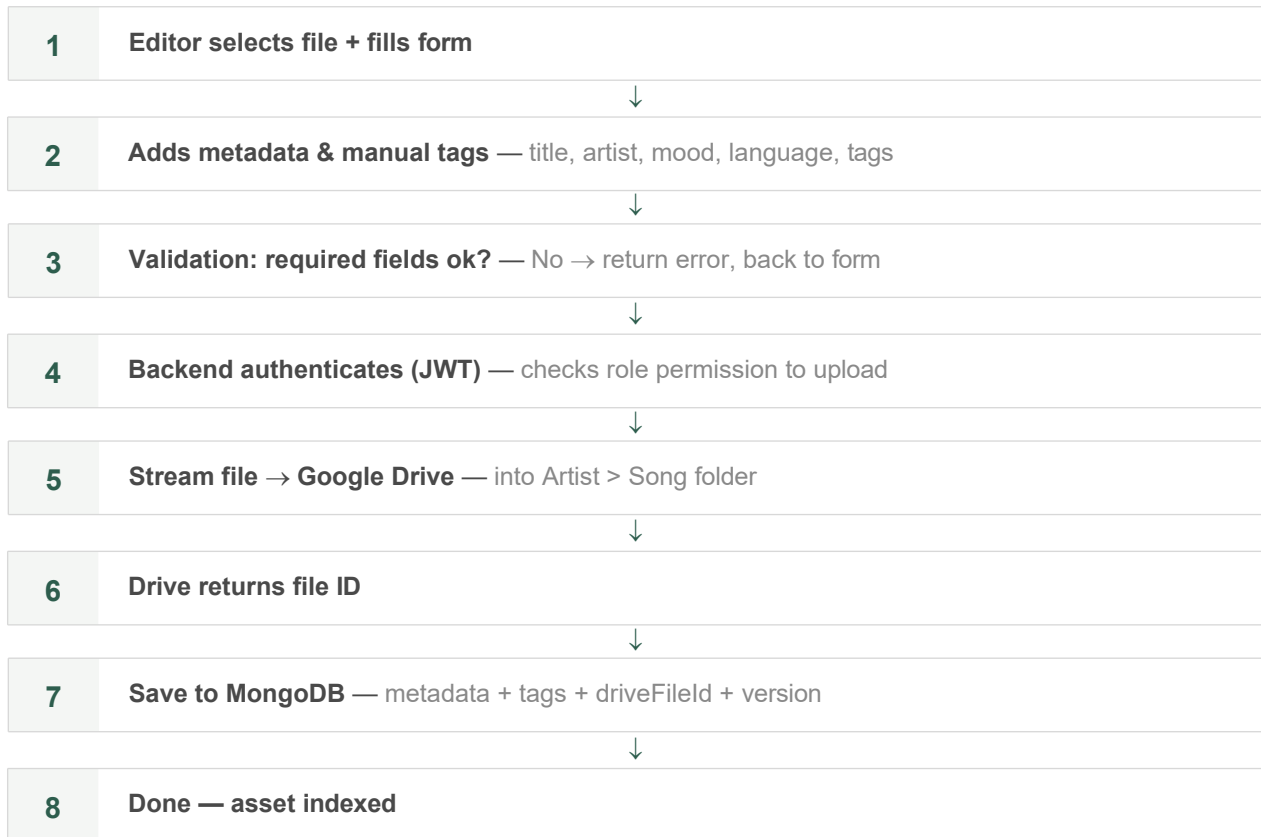
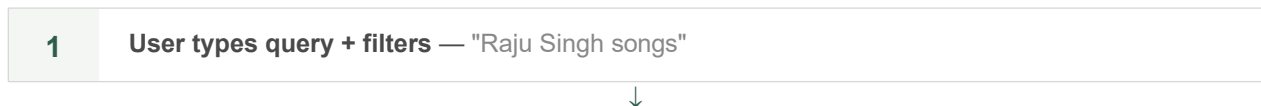


Figure 2 — Upload flow: validate → authenticate → store in Drive → index in MongoDB.

READ PATH

6. Search Flow

How a search query returns mixed-media results without the app ever touching the raw files.



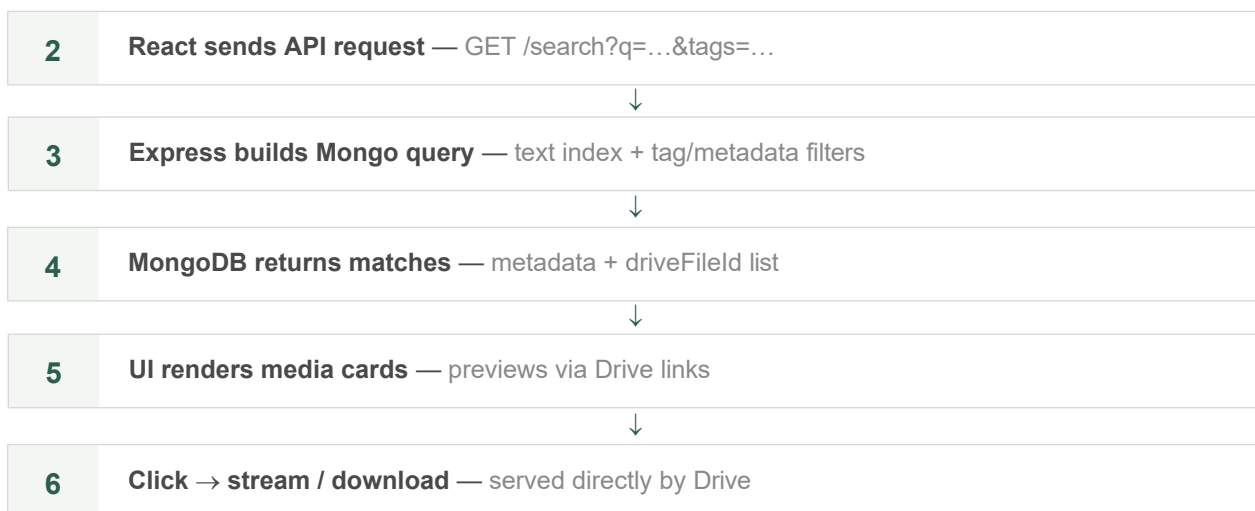


Figure 3 — Search flow: query MongoDB for metadata, then serve files straight from Drive.

THE ENGINE

7. Functional Modules

Ten cooperating modules make up the platform.

7.1 Artist Management

Manages every artist and links them to songs and assets: profiles, image galleries, genre, social links, collaborated songs.

7.2 Song Management

Manages all song-level data and links each song to its assets and versions. In MongoDB, a song is one document that embeds its metadata and an array of asset references.

7.3 Asset Management

The core media engine. Every asset connected to a song lives together in one place. Every uploaded asset is assigned exactly one type (Section 8), which drives filtering, folder placement, and the card icon shown in the UI.

7.4 Search & Discovery

A YouTube-like universal search across the whole library — returns audio, video, images, and artist assets instantly. Search by song, artist, genre, mood, language, release year, tags, or file type.

7.5 Manual Tagging

Editors tag assets by hand at upload (see Section 10). Tags drive search accuracy and stay intentional and clean.

7.6 Google Drive Integration

All files stored in and served from Drive via the Drive API using a service account. No heavy server storage; native streaming, preview, backup, and sharing.

7.7 Folder Structure Management

A predictable Drive hierarchy keeps files organized as the library scales:

Music Hub > Artist Name > Song Name > { Master Audio | Cover Art | Reel Clips | Lyrics | Promo Videos }

7.8 User Roles & Permissions

Role	Access
Admin	Full access; user and content management.
Editor	Upload content, edit metadata, manage tags.
Viewer	View and search content; limited downloads.
Marketing	Access promotional assets; share media externally.

7.9 Sharing System

Secure, controlled sharing: generate shareable and temporary (expiring) links, set per-link download permissions, track all access.

7.10 Version Management

Assets are replaced often. The system preserves a clear lineage — **V1, V2, Final, Final Master** — so teams never confuse old and updated files.

DELIVERABLES

8. Asset Type Catalogue

The exact list of deliverables the platform manages per song. Each upload is tagged with one **Type**. Types are grouped into four families so the UI can filter quickly and the Drive folders stay organized. Every type maps to a value of the `assets[].type` field in the song document (Section 9).

8.1 Video Deliverables

Type	Format	Description
Event Video	Horizontal	Footage from live shows, launches, and label events.
BTS of Song	H / V	Behind-the-scenes of the song's making (edited).
Review Video	Horizontal	Reviews, reactions, or commentary on the song.
Horizontal Video	16:9	General-purpose wide-format video (MV cuts, promos).
Reel — BTS / MV (+ Snippet)	9:16	Short social reel from BTS or the MV, carrying a song snippet.
Lyrics Video Reel	9:16	Reel with on-screen animated lyrics.
BTS — Unedited Footage	Raw	Raw source footage kept for re-editing; not for publishing as-is.

Type	Format	Description
Artist / Investor Reel on Song	9:16	Reel framed for the artist or investors highlighting the song.
Banner + Video	Mixed	Banner graphic paired with a short motion clip.

8.2 Cover & Graphic Deliverables

Type	Format	Description
Song Cover	Image	Primary cover artwork for the song.
Song Cover Square (+ Snippet)	1:1 video	Square cover animated with an audio snippet.
Song Cover Square (+ Snippet + Motion)	1:1 video	Square cover with audio snippet and motion graphics.
Banner Image	Image	Promotional banner for web, store, or social headers.
Post on Artist Page (Signature)	Image / Reel	Signature-style post formatted for the artist's own page.

8.3 Audio Deliverables

Type	Format	Description
Master Audio	WAV / MP3	Final mastered track.
Audio Snippet	MP3	Short clip used inside reels, covers, and teasers.
Song Snippet	MP3	Hook / teaser cut of the song for social promotion.

8.4 Text & Document Deliverables

Type	Format	Description
Lyrics	PDF / Text	Written lyrics document for the song.

WHY A FIXED CATALOGUE?

A controlled list of types (instead of free text) means every reel, cover, and banner is filed the same way every time — so filtering by ‘Reels’ or ‘Square Covers’ always returns the complete set, and folder structure stays predictable.

MONGODB COLLECTIONS

9. Data Model

A document-oriented design. The big win: a song embeds its metadata and an array of asset references, so loading a song and all its assets is a single read — no joins.

Collection	Key Fields	Notes
users	<code>_id, name, email, role, passwordHash</code>	Roles drive permissions
artists	<code>_id, name, label, genre, contact, imageFileId, socials[]</code>	Referenced by songs
songs	<code>_id, title, artistId, featuring[], language, mood, isrc, releaseDate, tags[], assets[]</code>	Embeds asset sub-documents
tags	<code>_id, name, type ("controlled" "custom")</code>	Source list for manual tagging
shares	<code>_id, assetId, token, expiresAt, canDownload</code>	Expiring share links
activityLog	<code>_id, userId, action, entity, timestamp</code>	Audit trail

9.1 Example Asset Metadata

Each asset is an embedded sub-document inside its song (the `assets[]` array). The fields below are drawn from the catalogue in Section 8 — one Type, one family, a format, and the `driveFileId` that links the metadata to the real file in Drive. Two examples follow: a video asset and an image asset.

```
// VIDEO asset - song_1042 > assets[]
{
  "type": "Reel - BTS/MV",
  "family": "Video",
  "format": "9:16",
  "driveFileId": "1C_qP...",
  "fileName": "dilse_bts_reel_v2.mp4",
  "mimeType": "video/mp4",
  "version": "V2",
  "durationSec": 28,
  "tags": ["Reel", "BTS", "Viral"],
  "uploadedBy": "user_12",
  "createdAt": "2026-02-10"
}
```

```
// IMAGE asset - song_1042 > assets[]
{
  "type": "Song Cover",
  "family": "Cover / Graphic",
  "format": "Image",
  "driveFileId": "1B_zW...",
  "fileName": "dilse_cover_3000x3000.jpg",
  "mimeType": "image/jpeg",
  "dimensions": "3000 x 3000",
  "version": "Final",
  "tags": ["Romantic", "Cover"],
  "uploadedBy": "user_08",
  "createdAt": "2026-02-08"
}
```

INDEXES

A text index on title + tags, plus single-field indexes on language, mood, artistId, and assets.type to keep faceted search and type-filtering fast.

BY PEOPLE, NOT ALGORITHMS

10. Manual Tagging System

Tagging is done by people, not algorithms. At upload (or later when editing), an Editor picks tags from a controlled list and may add custom tags. This keeps the vocabulary consistent while still flexible.

10.1 Controlled Tags (predefined)

A fixed set everyone shares, so the same concept is always spelled the same way:

Mood / Theme	Format / Use
Romantic, Sad, Party, Acoustic, Viral, Festive, Video Devotional	Promo, Reel, Teaser, BTS, Master, Demo, Lyric

10.2 Custom Tags (free-form)

Editors can add their own tags for context the controlled list doesn't cover (e.g. a campaign name or collaborator). Custom tags are stored with `type: "custom"` so they can be reviewed and, if useful, promoted into the controlled list later.

10.3 Why Manual

- **Accuracy** — a person knows the song's intent; no mis-tagging from a model.
- **Simplicity** — no ML pipeline to build, train, or maintain in V1.
- **Control** — the controlled list keeps tags clean and search reliable.

UX RULE

Good search depends on good tags. Make tag selection a required, friendly step in the upload form — controlled tags as clickable chips, with one optional 'add custom tag' field.

BUILD MATERIALS

11. Technology Stack

Layer	Technology	Reason
Frontend	React.js + Tailwind CSS	Fast SPA, reusable components, rapid styling
Backend	Node.js + Express.js	Lightweight REST API, strong Drive SDK support
Database	MongoDB (with Mongoose)	Document model fits songs + embedded assets
Storage	Google Drive API v3	Offloads media storage, streaming, backup
Auth	JWT	Stateless, role-aware authentication

Layer	Technology	Reason
Search	MongoDB text index + filters	Fast metadata- and tag-driven retrieval

11.1 Recommended Additions

- **Redis cache** — speed up popular searches and reuse Drive tokens.
- **Background job queue (BullMQ)** — handle large uploads and thumbnails asynchronously.
- **Atlas Search (optional)** — if MongoDB's basic text search outgrows needs later.

THE FEEL

12. UI / UX Concept

The interface blends the workflow clarity of **YouTube Studio**, the browsing comfort of a **Spotify dashboard**, and the visual appeal of **Netflix-style media cards**.

- Modern dark theme with accent colors cueing asset type (audio, video, image, doc).
- Fully responsive for desktop and tablet.
- Search-first home screen with a prominent universal search bar and quick filters.
- Media-card grid with inline previews; click-through to a song's connected-asset view.
- Consistent metadata panels for fast scanning and editing.

14 WEEKS

13. Development Roadmap

A phased plan that delivers a usable product early and layers extras later.

Phase	Focus	Key Deliverables	Duration
1	Foundation	Auth, roles, Mongo schemas, Drive integration	3 weeks
2	Core Modules	Artist, song & asset management	4 weeks
3	Search & Tags	Manual tagging, universal search	3 weeks
4	Collaboration	Sharing, versioning, activity logs	2 weeks
5	Polish	UI refinement, testing, deployment	2 weeks

13.1 Milestones

- **M1** — Working Drive upload/download with metadata saved to MongoDB.
- **M2** — Full CRUD for artists, songs, and assets.
- **M3** — Universal search returning mixed-media results.
- **M4** — Secure sharing and version history live.
- **M5** — Production deployment and user onboarding.

WHAT COULD BREAK

14. Risks & Mitigations

Risk	Impact	Mitigation
Drive API rate limits	Slow / failed uploads	Exponential backoff, batching, job queue
Large file timeouts	Failed transfers	Resumable uploads via Drive API
Inconsistent manual tags	Poor search results	Controlled tag list + required tag step
Unauthorized sharing	Asset leakage	Expiring links, access logs, role checks
Drive dependency	Vendor lock-in	Store file IDs + abstract a storage layer

PAYOFF

15. Benefits

15.1 Business Benefits

- Centralized media management and a single source of truth.
- Dramatically faster asset retrieval and fewer duplicate files.
- Improved cross-team collaboration and simpler partner sharing.

15.2 Technical Benefits

- Scalable, stateless architecture with reduced storage cost.
- Flexible document model that's easy to extend with new asset types.

FINAL VISION

Harmony Hub becomes the centralized digital music asset ecosystem where the company manages every song, artist media item, promotional file, and release asset from one intelligent dashboard — simplifying operations, strengthening collaboration, and laying a scalable foundation for future content management and distribution.